Excellent 🚀 Let's move on to **Module 18: Indexes**.

📘 SQL CLASS NOTE – MODULE 18: INDEXES

1. Introduction

An **Index** in SQL is a **database object** that improves the **speed of data retrieval**.

- Works like an **index in a textbook** → helps you find info quickly.
- Indexes are created on **columns** that are frequently used in:
  - `WHERE` clauses
  - `JOIN` conditions
  - `ORDER BY` or `GROUP BY`

👉 **Trade-off:**

- **Faster SELECT queries** ✅
- **Slower INSERT, UPDATE, DELETE** ❌ (because the index must be updated too).

2. Syntax

Create Index

```
CREATE INDEX index_name
ON table_name (column1, column2);
```

Drop Index

```
DROP INDEX index_name;
```

(Syntax varies slightly across databases; e.g., MySQL uses `ALTER TABLE DROP INDEX`.)

3. Types of Indexes

1. Single-Column Index

Index on **one column**.

```
CREATE INDEX idx_student_lastname
ON Students (LastName);
```

2. Composite Index

Index on **two or more columns**.

```
CREATE INDEX idx_student_name_grade
ON Students (LastName, GradeLevel);
```

3. Unique Index

Ensures **all values are unique** (often automatically created for `PRIMARY KEY`).

```
CREATE UNIQUE INDEX idx_unique_email
ON Students (Email);
```

4. Primary Key Index

- Automatically created when a primary key is defined.
- Example: `StudentID` in `Students` table.

5. Full-text Index *(DB-specific)*

Used for searching large text fields (e.g., article content).

4. Practical Examples with `SchoolDB`

Example 1: Speed up searches by LastName

```
CREATE INDEX idx_lastname
ON Students (LastName);
```

Now queries like:

```
SELECT * FROM Students WHERE LastName = 'Johnson';
```

will run **faster**.

Example 2: Improve queries filtering by Grade and Age

```
CREATE INDEX idx_grade_age
ON Students (GradeLevel, Age);
```

Usage:

```
SELECT * FROM Students
WHERE GradeLevel = 'Grade 12' AND Age > 16;
```

Example 3: Index for foreign key in Enrollments

```
CREATE INDEX idx_enrollments_student
ON Enrollments (StudentID);
```

This improves joins like:

```
SELECT S.FirstName, C.CourseName
FROM Students S
JOIN Enrollments E ON S.StudentID = E.StudentID
JOIN Courses C ON E.CourseID = C.CourseID;
```

5. Why Use Indexes?

- ✅ Faster searches, sorting, and filtering.
- ✅ Essential for large datasets.
- ✅ Optimizes joins.

👉 But:

- ❌ Slows down write operations (`INSERT` , `UPDATE` , `DELETE`).
- ❌ Takes up extra storage.

6. Real-Life Use Cases

- **Schools**: Search by StudentID or LastName quickly.
- **Banks**: Fast lookups of Account Number.
- **E-commerce**: Search products by name/category.
- **Hospitals**: Patient records by Medical ID.

7. Classwork (Practice) ✅

👉 Try creating these indexes:

1. An index on `LastName` in the **Students** table.
2. A composite index on `(GradeLevel, Age)` in **Students**.
3. An index on `StudentID` in **Enrollments** (to speed up joins).
4. A unique index on `Email` in **Students**.
5. Drop the index on `LastName` .

8. Challenge Question 💡

👉 Create an index that speeds up queries retrieving **courses taught by a specific teacher**, and test it with a query.

✅ That's the **expanded Module 18: Indexes**.

Perfect 👍 Let's solve the **Module 18 Classwork + Challenge** step by step with SQL queries.

✅ Solutions to Module 18: `INDEXES`

1. Create an index on `LastName` in the **Students** table.

```sql
CREATE INDEX idx_students_lastname
ON Students (LastName);
```

2. Create a composite index on `(GradeLevel, Age)` in **Students**.

```sql
CREATE INDEX idx_students_grade_age
ON Students (GradeLevel, Age);
```

3. Create an index on `StudentID` in **Enrollments** (to speed up joins).

```sql
CREATE INDEX idx_enrollments_student
ON Enrollments (StudentID);
```

4. Create a unique index on `Email` in **Students**.

```sql
CREATE UNIQUE INDEX idx_students_email
ON Students (Email);
```

5. Drop the index on `LastName`.

```
DROP INDEX idx_students_lastname;
```

(Note: In MySQL → `ALTER TABLE Students DROP INDEX idx_students_lastname;` *)*

💡 Challenge Question

👉 Create an index that speeds up queries retrieving **courses taught by a specific teacher**, and test it with a query.

Step 1: Create the index

```
CREATE INDEX idx_courses_teacher
ON Courses (Teacher);
```

Step 2: Use it in a query

```
SELECT CourseID, CourseName
FROM Courses
WHERE Teacher = 'Dr. Williams';
```

📌 The query will now use the index `idx_courses_teacher` to find the rows faster instead of scanning the whole table.